Floorplanning using Sequence Pairs

Given an input set of modules with their area and allowed aspect ratios, come up with a floorplan that minimises the area of the tightest box that encloses all the modules. Use simulated annealing to find a local optimum.

- Each module that is input is a instance of the class `Module`:

```
class Module:
  def __init__(self, name, area, aspect_ratios):
    self._name = name
    self._area = area
    self._wh = [(math.sqrt(area*r), math.sqrt(area/r)) for r in aspect_ratios]
  def __repr__(self):
    return f"'{self._name} area:{self._area} xy:{self._wh}'"
```

- Sequence pair and the choice of aspect ratio is stored as:

```
class SeqPair:
  def __init__(self, modules):
    self._pos = [i for i in range(len(modules))] # positive sequence
    self._neg = [i for i in range(len(modules))] # negative sequence
    self._ap  = [0 for i in range(len(modules))] # aspect ratio choice
    self._coords = [(0,0) for i in range(len(modules))]
    self._w = 0
    self._h = 0

  def perturb(self, modules):
  # fill in the perturbation function
    return None

  def costEval(self, modules):
  # create HCG, VCG, calculate the coordinates of all modules,
  # self._w and self._h of the floorplan and return the area
    return self._w * self._h
```

- Template for simulated annealing:

```
import math
import random

def accept(delC, T):
  if delC <= 0: return True
  return random.random() < math.exp(-delC/T)

# S = Initial sequence pair, choice of aspect ratio
def simulated_annelaing(Tmin, Tmax, N, alpha, S, modules, plot):
  assert(alpha < 1. and Tmin < Tmax)
  T = Tmax
  C = costEval(S)
  minC = C
  minS = S[:]
  Clist = []
  Temp = []
  while T > Tmin:
    for i in range(N):
      Snew = S.perturb(modules)
      Cnew = Snew.costEval(modules)
      if accept(Cnew - C, T):
        C, S = Cnew, Snew
        if minC >= Cnew:
          minC, minS = Cnew, Snew
        Clist.append(Cnew)
        Temp.append(T)
    T = T * alpha
  if plot:
    import matplotlib.pyplot as plt
    plt.plot(Temp, Clist)
    plt.xlim(max(Temp), min(Temp))
    plt.xscale('log')
  return minS, minC
```

- Top level function call to sequence pair floorplanner:

```
def sp_floorplan(modules):
  S = SeqPair(modules)
  Tmax = sum([i._area for i in modules])
  Smin, Cmin = simulated_annealing(1, Tmax, 100, 0.9, S, modules, False)
  assert(len(Smin._coord) == len(Smin._ap) and (len(Smin._coord) == len(modules)))
  sol = [(Smin._coord[i], m[i]._wh[Smin._ap[i]], m[i]._name) for i in range(len(modules)) ]
  return (sol, Cmin)
```

- Plotting utility:

```
def plot(coords):
  import matplotlib.pyplot as plt
  from matplotlib.patches import Rectangle

  fig, ax = plt.subplots()
  ax.plot([0, 0])
  ax.set_aspect('equal')
  ax.set_xlim(0,max([r[0][0] + r[1][0] for r in coords]))
  ax.set_ylim(0,max([r[0][1] + r[1][1] for r in coords]))

  for i,r in enumerate(coords):
    match i%4:
      case 3: hatch, color = '/+', 'red'
      case 2: hatch, color = '///', 'green'
      case 1: hatch, color = '/\//\//\/', 'blue'
      case _: hatch, color = '\\', 'gray'
    ax.add_patch(Rectangle(r[0], r[1][0], r[1][1],
                edgecolor = color, facecolor=color,
                hatch=hatch, fill = False,
                lw=2))
    ax.text(r[0][0] + r[1][0]//2, r[0][1] + r[1][1]//2, r[2], fontsize=8)
  plt.show()
```

- Example function call with four modules:

```
m = [Module('a', 16, [0.25, 4]), Module('b', 32, [2.0, 0.5]), Module('c', 27, [1./3, 3.]), \
                Module('d', 6, [6])]
sol, area = sp_floorplan(m, 0.75, 1.33)
plot(sol)
```

- Example floorplan call with random modules:

```
m = [Module(str(i), random.randint(10,100), [1.]) for i in range(10)]
sol, area = sp_floorplan(m)
plot(sol)
```

Use the Python notebook below for reference:
https://github.com/srini229/EE5333_tutorials/blob/master/fp/Floorplanning.ipynb